End-to-End ML Experiment: Heart Disease Prediction

An end-to-end machine learning pipeline for predicting heart disease risk using the UCI Heart Disease dataset. This project demonstrates modern MLOps practices including data processing, model development, experiment tracking, packaging, testing, and monitoring.

Team 70 - Contributors

Name	Email Address	Contributions %
NEERAJ BHATT	2024aa05020@wilp.bits-pilani.ac.in	100%
V. S. BALAKRISHNAN	2024aa05017@wilp.bits-pilani.ac.in	100%
AVANISH KUMAR SINGH	2024aa05353@wilp.bits-pilani.ac.in	100%
SAJAL CHAUDHARY	2024aa05026@wilp.bits-pilani.ac.in	100%
SACHIN KUMAR	2024aa05024@wilp.bits-pilani.ac.in	100%

Project Overview

This project implements a complete ML lifecycle for binary classification of heart disease presence/absence based on patient health data. The pipeline includes:

- Data acquisition and exploratory data analysis
- Feature engineering and model development
- Experiment tracking with MLflow
- Model packaging and reproducibility
- Automated testing and CI/CD
- Logging and monitoring

Dataset

Source: UCI Machine Learning Repository - Heart Disease Dataset

- **Size**: 303 samples, 13 features
- **Target**: Binary classification (heart disease present/absent)
- **Features**: Age, sex, chest pain type, blood pressure, cholesterol, etc.

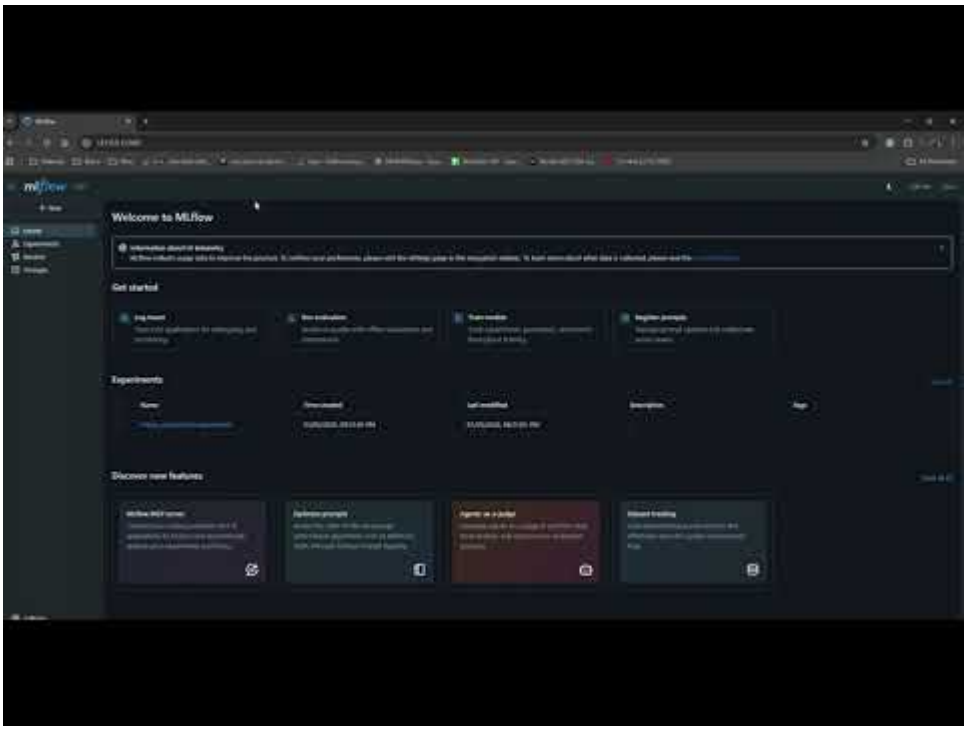
Demo Video

A demonstration video showing the end-to-end pipeline execution is available:

Direct YouTube Link

🎬 Watch on YouTube

Thumbnail Link



Alternative Video Formats

- **MP4 Download**: Download Video
- **GitHub Repository**: View in Repository

Content: Complete workflow from data processing to model deployment
Duration: ~5-10 minutes
Topics Covered: Data processing, model training, MLflow tracking, Docker containerization, Kubernetes deployment

Setup Instructions

Prerequisites

- Python 3.11+
- Git

Local Setup

1. **Clone the repository**:

```
git clone https://github.com/balakrishnanvinchu/e2e-ml-experiment.git
cd e2e-ml-experiment
```

2. **Create virtual environment**:

```
python -m venv .venv
# On Windows:
.venv\Scripts\activate
```

```
# On macOS/Linux:
source .venv/bin/activate
```

3. **Install dependencies**:

```
pip install -r requirements.txt
```

4. **Run tests**:

```
python -m pytest tests/ -v
```

5. **Make predictions**:

```
python predict.py
```

Project Structure

```
├── data/                          # Dataset files
│   ├── cleaned_data.csv          # Preprocessed dataset
│   ├── X_features.csv            # Feature matrix
│   └── y_target.csv              # Target labels
├── models/                        # Saved models and artifacts
│   ├── best_model_logistic_regression.pkl
│   ├── scaler.pkl
│   ├── feature_names.json
│   └── model_metrics.json
├── reports/                       # Experiment reports
│   ├── mlflow_experiment_summary.json
│   └── mlflow_runs.csv
├── logs/                         # Prediction logs
├── tests/                        # Unit tests
│   ├── test_preprocess.py
│   └── test_predict.py
├── .github/workflows/            # CI/CD workflows
│   └── ci.yml
├── 01_data_acquisition_eda.ipynb          # Data processing notebook
├── 02_feature_engineering_model_development.ipynb  # Model training
├── 03_model_tracking_experiments.ipynb    # MLflow tracking
├── app.py                         # Flask API application
├── preprocess.py                  # Data preprocessing script
├── predict.py                     # Prediction script
├── Dockerfile                     # Docker container definition
├── .dockerignore                  # Docker ignore file
├── requirements.txt               # Python dependencies (full)
├── requirements-docker.txt        # Docker-specific dependencies
```

```
├── k8s-deployment.yaml           # Kubernetes deployment manifest
├── k8s-service.yaml              # Kubernetes service manifest
├── k8s-deploy-instructions.md    # K8s deployment guide
├── MLOPS_Report.md               # Detailed project report
└── README.md                      # This file
```

Model Details

Best Model: Logistic Regression

- **Accuracy**: 60.66%
- **F1-Score**: 58.39%
- **Features**: 13 standardized features
- **Preprocessing**: Standard scaling

Model Performance

- Cross-validation mean accuracy: ~67%
- Test accuracy: 60.7%
- No significant overfitting detected

API Usage

Prediction Endpoint

The prediction script accepts JSON input with patient features:

```python
from predict import predict_heart_disease

# Sample input
patient_data = {
    "age": 63,
    "sex": 1,
    "cp": 3,
    "trestbps": 145,
    "chol": 233,
    "fbs": 1,
    "restecg": 0,
    "thalach": 150,
    "exang": 0,
    "oldpeak": 2.3,
    "slope": 0,
    "ca": 0,
    "thal": 1
}

result = predict_heart_disease(patient_data)
print(result)
```

Output:


```
{
  "prediction": 0,
  "prediction_label": "No Heart Disease",
  "confidence": 0.948,
  "probabilities": {
    "no_disease": 0.948,
    "disease": 0.052
  }
}
```


Experiment Tracking

Experiments are tracked using MLflow. To view experiments:

```
mlflow ui
```

Navigate to `http://localhost:5000` to explore runs, metrics, and artifacts.

Testing

Run the test suite:

```
python -m pytest tests/ -v
```

Tests cover:

- Data preprocessing functionality
- Prediction output validation
- Model consistency

CI/CD

GitHub Actions workflow includes:

- Dependency installation
- Code linting with flake8
- Unit test execution
- Model training (on main branch pushes)

Monitoring & Logging

- Prediction requests and results are logged to `logs/predictions.log`
- Logs include timestamps, processing time, and full request/response data

- Error handling with detailed error logging

Architecture Diagram

```
Raw Data → Data Cleaning → Feature Engineering → Model Training → MLflow Tracking
     ↓                                                              ↓
Model Packaging → Unit Tests → CI/CD Pipeline → Prediction API → Logging
```

Future Enhancements

- Cloud deployment (Azure/AWS)
- Real-time monitoring dashboard
- Model retraining pipeline
- API authentication and rate limiting

Docker Containerization

The application is containerized using Docker for easy deployment and scaling.

Build Docker Image

```
docker build -t heart-disease-predictor:v1.0 .
```

Run Locally

```
docker run -d -p 5000:5000 --name heart-disease-api heart-disease-predictor:v1.0
```

Test Container

```
# Health check
curl http://localhost:5000/health

# Prediction
curl -X POST http://localhost:5000/predict \
  -H "Content-Type: application/json" \
  -d '{"age": 63, "sex": 1, "cp": 3, "trestbps": 145, "chol": 233, "fbs": 1,
"restecg": 0, "thalach": 150, "exang": 0, "oldpeak": 2.3, "slope": 0, "ca": 0,
"thal": 1}'
```

Kubernetes Deployment

The application can be deployed to local Kubernetes using Docker Desktop or Minikube.

Prerequisites

- **Docker Desktop**: Enable Kubernetes in Settings > Kubernetes
- **Minikube**: Install and run `minikube start`

Deploy to Kubernetes

1. **Ensure cluster is running**:

```
kubectl cluster-info
```

2. **Load image into cluster** (for Minikube):

```
eval $(minikube docker-env)
docker build -t heart-disease-predictor:v1.0 .
```

3. **Deploy application**:

```
kubectl apply -f k8s-deployment.yaml
kubectl apply -f k8s-service.yaml
```

4. **Check status**:

```
kubectl get pods
kubectl get services
```

5. **Get service URL**:
   - **Minikube**: `minikube service heart-disease-service --url`
   - **Docker Desktop**: Check service external IP

Scaling

```
kubectl scale deployment heart-disease-predictor --replicas=3
```

Monitoring

```
kubectl logs -l app=heart-disease-predictor
kubectl describe pods
```

Contributing

1. Fork the repository
2. Create a feature branch
3. Make changes with tests
4. Submit a pull request

License

This project is licensed under the Apache License - see the LICENSE file for details.

Contact

For questions or issues, please open a GitHub issue.